

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Class Test)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO3-AT2-CLASS TEST
Question	Q6 — List Aliasing and the id() Function (1 Mark)

Q6. [1 Mark]

Explain with Python code how aliasing works with lists. Show how modifying an aliased list changes the original, and explain the reason using id() function.

Explanation

In Python, a list variable does not store the list's contents directly — it stores a reference (a memory address) to a list object. When one list variable is assigned to another (e.g. `alias = original`), no new list is created; both variable names simply point to the **same** underlying object. This is called **aliasing**. Because both names refer to the identical object, changing the list through one name — for example appending an item via `alias.append(40)` — is visible through the other name as well, since there is only ever one list in memory. The built-in `id()` function returns the unique memory identifier of an object, so printing `id(original)` and `id(alias)` and finding them equal confirms that both variables reference the same list object rather than independent copies.

Python Program

```
"""
CO3-AT2 - Class Test
Q6: List Aliasing and the id() Function [1 Mark]
-----
Demonstrates how aliasing works with lists: assigning one list
variable to another does not create a copy, so modifying the
"aliased" list also changes the original. Uses id() to show that
both variables refer to the same object in memory.
"""

# Original list
original = [10, 20, 30]

# Aliasing: alias now refers to the SAME list object as original,
# not a new copy of it
alias = original

print("Before modification:")
print("original:", original)
print("alias:   ", alias)
print("id(original):", id(original))
print("id(alias):   ", id(alias))
print("Same object? ", id(original) == id(alias))

# Modifying the aliased list
alias.append(40)
print("\nAfter alias.append(40):")
```

```
print("original:", original) # original also changes!
print("alias: ", alias)
print("id(original):", id(original))
print("id(alias): ", id(alias))
```

Sample Output

```
Before modification:
original: [10, 20, 30]
alias:    [10, 20, 30]
id(original): 139636700957504
id(alias):    139636700957504
Same object? True
```

```
After alias.append(40):
original: [10, 20, 30, 40]
alias:    [10, 20, 30, 40]
id(original): 139636700957504
id(alias):    139636700957504
```

Note: the exact numeric `id()` values will differ between runs and machines, but the key result is that `id(original)` and `id(alias)` always match each other, both before and after the modification.

Conclusion

The program shows that `alias = original` creates a second name for the same list object rather than an independent copy, which `id()` confirms by returning identical memory addresses for both names.

Consequently, appending 40 through `alias` also updates `original` to `[10, 20, 30, 40]`, demonstrating why aliasing must be handled carefully — a true independent copy would instead require `original.copy()` or `list(original)`.